Spring AI 1.1.3 RAG ETL PDF 项目 Demo

以下基于 Spring AI **1.1.3** 版本，构建一个完整的 PDF RAG ETL 流水线项目。Spring AI 1.1.3 是该框架的一个重要版本，发布于 2026 年 3 月，包含 19 个新特性和 31 个 Bug 修复。

一、前置条件与环境准备

- **JDK 17+**：Spring Boot 3.x 和 Spring AI 1.1.3 基于较新的 Java 版本
- **Maven 或 Gradle**：本示例以 Maven 为例
- **OpenAI API Key**（或其他兼容的 Embedding/Chat 模型服务商）

二、项目依赖配置（pom.xml）

Spring AI 1.1.3 使用 BOM（Bill of Materials）进行依赖版本管理，统一管理所有 Spring AI 相关依赖的版本。

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.5.0</version>
        <relativePath/>
    </parent>

    <groupId>com.example</groupId>
    <artifactId>spring-ai-rag-pdf-demo</artifactId>
    <version>1.0.0</version>
    <name>Spring AI RAG PDF Demo</name>
    <description>RAG ETL PDF pipeline with Spring AI 1.1.3</description>

    <properties>
        <java.version>17</java.version>
        <spring-ai.version>1.1.3</spring-ai.version>
    </properties>

    <dependencyManagement>
        <dependencies>
            <!-- Spring AI BOM：统一管理所有 Spring AI 相关依赖的版本 -->
            <dependency>
                <groupId>org.springframework.ai</groupId>
                <artifactId>spring-ai-bom</artifactId>
                <version>${spring-ai.version}</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
```

```
        </dependencies>
    </dependencyManagement>

    <dependencies>
        <!-- Spring Boot Web Starter -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <!-- Spring AI OpenAI Starter（包含 ChatModel 和 EmbeddingModel） -->
        <dependency>
            <groupId>org.springframework.ai</groupId>
            <artifactId>spring-ai-openai-spring-boot-starter</artifactId>
        </dependency>

        <!-- Spring AI PDF Document Reader：用于解析 PDF 文件 -->
        <dependency>
            <groupId>org.springframework.ai</groupId>
            <artifactId>spring-ai-pdf-document-reader</artifactId>
        </dependency>

        <!-- Spring AI Tika Document Reader（可选，支持更多文档格式） -->
        <dependency>
            <groupId>org.springframework.ai</groupId>
            <artifactId>spring-ai-tika-document-reader</artifactId>
        </dependency>
    </dependencies>

    <repositories>
        <!-- Spring Milestones 仓库（Spring AI 1.1.3 发布在 Milestone 仓库） -->
        <repository>
            <id>spring-milestones</id>
            <name>Spring Milestones</name>
            <url>https://repo.spring.io/milestone</url>
            <snapshots>
                <enabled>false</enabled>
            </snapshots>
        </repository>
    </repositories>
</project>
```

> **说明**：Spring AI 1.1.3 的 artifacts 发布在 Spring Milestone 仓库中，因此需要配置该仓库才能下载依赖。

三、配置文件（application.yml）

在 `src/main/resources/application.yml` 中配置 OpenAI API 连接信息：

```yaml
spring:
  ai:
    openai:
      api-key: ${OPENAI_API_KEY:your-api-key-here}
      # 如需使用 DeepSeek 等兼容 OpenAI API 的服务，可配置 base-url
      # base-url: https://api.deepseek.com
      embedding:
```

```yaml
        options:
          model: text-embedding-ada-002
      chat:
        options:
          model: gpt-3.5-turbo

logging:
  level:
    com.example: DEBUG
    org.springframework.ai: INFO
```

> **注意**：Spring AI 1.1.3 支持通过 `spring.ai.openai.embedding.options.model` 配置 embedding 模型，也支持自定义 embedding 维度。

四、核心代码实现

4.1 文档加载与 ETL 处理服务

```java
package com.example.rag.service;

import jakarta.annotation.PostConstruct;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.ai.document.Document;
import org.springframework.ai.document.DocumentReader;
import org.springframework.ai.embedding.EmbeddingModel;
import org.springframework.ai.reader.pdf.PagePdfDocumentReader;
import org.springframework.ai.transformer.splitter.TokenTextSplitter;
import org.springframework.ai.vectorstore.SimpleVectorStore;
import org.springframework.ai.vectorstore.VectorStore;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.core.io.Resource;
import org.springframework.stereotype.Service;

import java.util.List;

/**
 * PDF ETL 处理服务
 * 负责从 PDF 文档中提取文本、分割、向量化并存储到向量数据库
 */
@Service
public class PdfEtlService {

    private static final Logger logger =
LoggerFactory.getLogger(PdfEtlService.class);

    private final VectorStore vectorStore;
    private final EmbeddingModel embeddingModel;

    @Value("classpath:docs/sample.pdf")
    private Resource pdfResource;

    public PdfEtlService(EmbeddingModel embeddingModel) {
        this.embeddingModel = embeddingModel;
        // 使用 SimpleVectorStore 作为内存向量存储（Spring AI 1.1.3 支持）
```

```java
        this.vectorStore = SimpleVectorStore.builder(embeddingModel).build();
    }

    /**
     * 应用启动后自动加载 PDF 并执行 ETL 流水线
     */
    @PostConstruct
    public void loadPdfDocuments() {
        try {
            logger.info("Starting PDF ETL pipeline...");

            // 1. Extract - 使用 PagePdfDocumentReader 读取 PDF（按页分割，每页一个
Document）
            DocumentReader pdfReader = new PagePdfDocumentReader(pdfResource);
            List<Document> documents = pdfReader.read();
            logger.info("Extracted {} pages from PDF", documents.size());

            // 2. Transform - 使用 TokenTextSplitter 将文档分割成适合 embedding 的
chunks
            TokenTextSplitter splitter = TokenTextSplitter.builder()
                    .withChunkSize(800)          // 每个 chunk 最大 token 数
                    .withChunkOverlap(100)        // 相邻 chunk 之间的重叠 token 数
                    .withMinChunkSizeChars(350)   // 最小 chunk 字符数
                    .withEmbeddingModel(embeddingModel)
                    .build();
            List<Document> splitDocuments = splitter.split(documents);
            logger.info("Split into {} chunks", splitDocuments.size());

            // 3. Load - 写入向量存储
            // Domain-specific 风格写法（Spring AI 推荐）
            vectorStore.write(splitDocuments);
            logger.info("Successfully loaded {} documents into vector store",
splitDocuments.size());

        } catch (Exception e) {
            logger.error("Failed to execute PDF ETL pipeline", e);
        }
    }

    /**
     * 获取 VectorStore 实例供查询使用
     */
    public VectorStore getVectorStore() {
        return vectorStore;
    }
}
```

> **关于 TokenTextSplitter**：在 Spring AI 1.1.3 中，`TokenTextSplitter` 支持链式 builder 模式配置 `chunkSize`、`chunkOverlap` 和 `minChunkSizeChars` 参数。`chunkSize` 指定每个 chunk 的最大 token 数（默认 800），`overlap` 保留相邻 chunk 之间的上下文连续性。

4.2 RAG 问答服务

```java
package com.example.rag.service;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.ai.chat.client.ChatClient;
import org.springframework.ai.chat.client.advisor.QuestionAnswerAdvisor;
import org.springframework.ai.vectorstore.VectorStore;
import org.springframework.stereotype.Service;

/**
 * RAG 问答服务
 * 基于已加载的向量存储执行检索增强生成
 */
@Service
public class RagService {

    private static final Logger logger =
LoggerFactory.getLogger(RagService.class);

    private final ChatClient chatClient;
    private final VectorStore vectorStore;

    public RagService(ChatClient.Builder chatClientBuilder, PdfEtlService
pdfEtlService) {
        this.vectorStore = pdfEtlService.getVectorStore();
        // 构建 ChatClient 并注册 QuestionAnswerAdvisor 实现 RAG
        this.chatClient = chatClientBuilder
                .defaultAdvisors(new QuestionAnswerAdvisor(vectorStore))
                .build();
    }

    /**
     * 基于 RAG 回答用户问题
     * @param question 用户问题
     * @return AI 生成的回答
     */
    public String ask(String question) {
        logger.info("RAG query: {}", question);
        return chatClient.prompt()
                .user(question)
                .call()
                .content();
    }

    /**
     * 仅检索相关文档，不调用 LLM 生成回答（用于调试）
     */
    public List<String> searchRelevantDocuments(String query, int topK) {
        return vectorStore.similaritySearch(
                org.springframework.ai.vectorstore.SearchRequest.query(query)
                        .withTopK(topK))
                .stream()
                .map(doc -> doc.getContent())
                .collect(Collectors.toList());
    }
```

```
}
```

> **关于 QuestionAnswerAdvisor**：Spring AI 1.1.3 使用 Advisor API 为常见的 RAG 流程提供开箱即用的支持。`QuestionAnswerAdvisor` 会自动从 VectorStore 中检索相关文档并增强 prompt。

4.3 REST 控制器

```java
package com.example.rag.controller;

import com.example.rag.service.RagService;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Map;

@RestController
@RequestMapping("/api/rag")
public class RagController {

    private final RagService ragService;

    public RagController(RagService ragService) {
        this.ragService = ragService;
    }

    @PostMapping("/ask")
    public Map<String, String> ask(@RequestBody QuestionRequest request) {
        String answer = ragService.ask(request.question());
        return Map.of("question", request.question(), "answer", answer);
    }

    @GetMapping("/search")
    public List<String> search(@RequestParam String query,
                               @RequestParam(defaultValue = "3") int topK) {
        return ragService.searchRelevantDocuments(query, topK);
    }

    public record QuestionRequest(String question) {}
}
```

4.4 启动类

```java
package com.example.rag;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class RagPdfApplication {
    public static void main(String[] args) {
        SpringApplication.run(RagPdfApplication.class, args);
    }
}
```

五、ETL 流水线核心组件说明

Spring AI 的 ETL 流水线遵循模块化架构，分为三个核心阶段：

阶段	接口	本示例实现	说明
Extract	`DocumentReader`	`PagePdfDocumentReader`	按页读取 PDF，每页生成一个 `Document` 对象，保留页码、文件路径等元数据
Transform	`DocumentTransformer`	`TokenTextSplitter`	基于 token 数量进行文本分割，确保每个 chunk 不超过 embedding 模型的上下文窗口
Load	`DocumentWriter` / `VectorStore`	`SimpleVectorStore`	将 Document 向量化后写入向量存储。SimpleVectorStore 支持 JSON 文件持久化

两种 ETL 写法风格（Spring AI 1.1.3 均支持）：

```
// 函数式风格
vectorStore.accept(tokenTextSplitter.apply(pdfReader.get()));

// Domain-specific 风格（推荐）
vectorStore.write(tokenTextSplitter.split(pdfReader.read()));
```

六、高级扩展选项

6.1 使用 TikaDocumentReader 支持多种文档格式

如果项目需要处理多种文档格式（PDF、Word、PPT、HTML 等），可以使用基于 Apache Tika 的 `TikaDocumentReader`：

```
import org.springframework.ai.reader.tika.TikaDocumentReader;

Resource resource = new FileSystemResource("/path/to/document.docx");
TikaDocumentReader reader = new TikaDocumentReader(resource);
List<Document> documents = reader.read();
```

6.2 使用外部向量数据库

Spring AI 1.1.3 支持多种向量数据库实现，包括 **ChromaVectorStore**、**PgVectorStore**、**RedisVectorStore**、**Neo4jVectorStore** 等。

以 Redis 向量存储为例，添加依赖并配置 Bean：

```
<dependency>
    <groupId>org.springframework.ai</groupId>
    <artifactId>spring-ai-redis-spring-boot-starter</artifactId>
</dependency>
```

```java
@Bean
public VectorStore vectorStore(EmbeddingModel embeddingModel) {
    return RedisVectorStore.builder()
            .embeddingModel(embeddingModel)
            .indexName("pdf-index")
            .build();
}
```

Spring AI 1.1.3 中 Neo4j 向量存储还支持通过 builder 模式自定义过滤表达式转换器，提供更灵活的查询过滤逻辑。

6.3 添加 Metadata 增强

可以在 Transformer 阶段添加 `KeywordMetadataEnricher` 或 `SummaryMetadataEnricher` 来丰富文档元数据，提升检索效果：

```java
KeywordMetadataEnricher enricher = KeywordMetadataEnricher.builder()
        .chatModel(chatModel)
        .keywordCount(5)
        .build();
List<Document> enrichedDocs = enricher.apply(splitDocuments);
```

6.4 持久化 SimpleVectorStore

`SimpleVectorStore` 支持将向量数据保存到 JSON 文件，便于后续加载复用：

```java
// 保存向量存储到文件
((SimpleVectorStore) vectorStore).save(new File("vector-store.json"));

// 从文件加载
SimpleVectorStore vectorStore =
SimpleVectorStore.builder(embeddingModel).build();
vectorStore.load(new File("vector-store.json"));
```

七、测试与运行

7.1 准备测试 PDF

在 `src/main/resources/docs/` 目录下放置一个测试 PDF 文件（如 `sample.pdf`）。

7.2 启动应用

```bash
# 设置环境变量
export OPENAI_API_KEY=your-api-key

# 启动 Spring Boot 应用
mvn spring-boot:run
```

7.3 测试 API

```
# RAG 问答
curl -X POST http://localhost:8080/api/rag/ask \
  -H "Content-Type: application/json" \
  -d '{"question": "PDF文档中提到了哪些关键内容？"}'

# 检索相关文档（不生成回答）
curl "http://localhost:8080/api/rag/search?query=关键内容&topK=3"
```

八、项目结构总结

```
spring-ai-rag-pdf-demo/
├── pom.xml
├── src/
|   ├── main/
|   |   ├── java/
|   |   |   └── com/example/rag/
|   |   |       ├── RagPdfApplication.java      # 启动类
|   |   |       ├── controller/
|   |   |       |   └── RagController.java       # REST 接口
|   |   |       └── service/
|   |   |           ├── PdfEtlService.java       # PDF ETL 处理
|   |   |           └── RagService.java          # RAG 问答
|   |   └── resources/
|   |       ├── application.yml                  # 配置文件
|   |       └── docs/
|   |           └── sample.pdf                   # 测试 PDF 文档
|   └── test/
|       └── java/...
```

九、注意事项

1. **版本严格对应**：以上所有代码和配置均基于 Spring AI **1.1.3** 版本编写，与其他版本（如 1.0.0-M1）的 API 可能存在差异，使用时请确保 BOM 版本设置为 `1.1.3`。
2. **Spring Milestone 仓库**：Spring AI 1.1.3 尚未发布到 Maven Central，需要在 `pom.xml` 中配置 Spring Milestones 仓库才能下载依赖。
3. **PDF 解析依赖**：`spring-ai-pdf-document-reader` 内部依赖 Apache PDFBox，无需额外配置。
4. **Embedding 成本**：每次调用 OpenAI Embedding API 都会产生费用，生产环境建议考虑本地 embedding 模型（如通过 Ollama 集成）。
5. **Spring AI 1.1.3 新特性**：该版本新增了 `SimpleVectorStore` 的删除过滤功能、`OpenAiSdkChatModel` 的 builder 模式等特性，构建项目时可充分利用。